

Mastering LangChain: Chains, Pipelines, and Pythonic Operator Overloading

Linux Pipe Operator (|)

The pipe operator | connects two Linux commands.

The **output of the left command becomes the input to the right command.**

cmd1 | cmd2 | cmd3 | ...

Example

cat students.txt | grep "Python"

students.txt

Ramesh Python

Suresh Java

Mahesh Python

Shiva Devops

Command:

cat students.txt | grep "Python"

Output:

Ramesh Python

Mahesh Python

We can connect one more command:

cat students.txt | grep "Python" | wc -l

Output:

2

Python Operator Overloading

Python allows us to give special meaning to operators for our own objects.

Example:

class Employee:

```
def __init__(self, name, salary):
```

```
    self.name = name
```

```
    self.salary = salary
```

```
def __add__(self, other):
```

```
    return self.salary + other.salary
```

```
e1 = Employee("Durga", 10000)
```

```
e2 = Employee("Shiva", 20000)
```

```
e3 = Employee("Ravi", 30000)
```

```
# print(10 + 20)
```

```
# print("durga" + "soft")
```

```
print(e1 + e2)    # e1.__add__(e2)
```

Here:

```
e1 + e2
```

internally means:

```
e1.__add__(e2)
```

Adding Multiple Employee Objects

class Employee:

```
def __init__(self, name, salary):
```

```
    self.name = name
```

```
    self.salary = salary
```

```
def __add__(self, other):
```

```
    total = self.salary + other.salary
```

```
    return Employee("XYZ", total)
```

```
e1 = Employee("Durga", 10000)
```

```
e2 = Employee("Shiva", 20000)
```

```
e3 = Employee("Ravi", 30000)
```

```
# print(10 + 20)
```

```
# print("durga" + "soft")
```

```
print(e1 + e2)    # e1.__add__(e2)
```

```
e = e1 + e2 + e3
```

```
print(e.salary)  # 60000
```

| Operator in Python

| is the **bitwise OR operator**.

For sets, | works as the **union operator**.

s1 | s2

is equivalent to:

s1.union(s2)

To overload the | operator, we have to implement the magic method:

__or__()

Overloading | Operator

class Employee:

```
def __init__(self, name, salary):
```

```
    self.name = name
```

```
    self.salary = salary
```

```
def __add__(self, other):
```

```
    total = self.salary + other.salary
```

```
    return Employee("XYZ", total)
```

```
def __or__(self, other):
```

```
    return self.salary - other.salary
```

```
e1 = Employee("Durga", 30000)
```

```
e2 = Employee("Shiva", 20000)
```

```
e3 = Employee("Ravi", 10000)
```

```
print(e1 | e2)
```

LangChain — Manual Approach

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(
```

```
    "Explain {topic} in very simple English."
```

```
)
```

```
model = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
parser = StrOutputParser()
```

```
prompt_value = prompt.invoke(
```

```
    {"topic": "Python"}
```

```
)
```

```
response = model.invoke(prompt_value)
```

```
result = parser.invoke(response)
```

```
print(result)
```

Here we are manually executing every component one after another:

Prompt



Model



Parser

LangChain — Chain Approach

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(  
    "Explain {topic} in very simple English."  
)
```

```
model = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")
```

)

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```
response = chain.invoke(  
    {"topic": "Java"}  
)
```

```
print(response)
```

The chain is:

```
prompt | model | parser
```

Flow:

Input



PromptTemplate



ChatOpenAI



StrOutputParser



Final Output

CCE

Remember the three steps:

CCE

C → Creation of Components

Create the required components:

prompt

model

parser

C → Creation of Chain

Connect the components using the | operator:

chain = prompt | model | parser

E → Execute the Chain

Execute the chain by passing input to the first component:

```
response = chain.invoke(  
    {"topic": "Java"}  
)
```

So:

C → Creation of Components

C → Creation of Chain

E → Execute the Chain